

Caso práctico 1 — UD 4

Creación, uso y manejo de matrices (arrays) en JavaScript

1. Introducción

El presente caso práctico se centra en la gestión de datos en el entorno cliente mediante el uso de matrices en JavaScript. En lugar de trabajar con valores aislados, la aplicación organiza la información en una estructura de objetos almacenados en un array, lo que permite tratar el inventario como un conjunto dinámico y manipulable de datos.

A partir de esta estructura, se implementan operaciones típicas de gestión (alta, modificación, eliminación y filtrado) utilizando métodos propios de las matrices como *map*, *filter* y *reduce*. Estos métodos permiten transformar, seleccionar y acumular información de forma declarativa, evitando bucles tradicionales y favoreciendo un código más claro y mantenible.

La interfaz interactúa directamente con el DOM, actualizando la representación visual de los datos cada vez que el array cambia, sin recargar la página. De este modo, la aplicación no solo cumple una función práctica de inventario, sino que demuestra el papel central de las matrices en la construcción de aplicaciones web dinámicas.

2. Definición y estructura de la matriz principal

La lógica de la aplicación se apoya en una matriz denominada *productos*, que actúa como fuente única de datos del inventario. Cada elemento del array es un objeto con tres propiedades: *nombre*, *precio* y *cantidad*. De esta manera, cada producto se representa como una entidad independiente dentro de una estructura común.

Ejemplo de definición inicial:

```
let productos = [
  { nombre: "Teclado", precio: 39.99, cantidad: 10 },
  { nombre: "Ratón", precio: 19.99, cantidad: 25 },
  { nombre: "Monitor", precio: 149.99, cantidad: 6 },
];
```

Cada elemento del array encapsula la información necesaria para calcular el valor individual del producto (precio × cantidad) y para mostrar sus datos en la interfaz. Esta estructura permite tratar el inventario como un conjunto dinámico de objetos que pueden añadirse, modificarse o eliminarse sin alterar el resto del sistema.

El uso de una única matriz como estado de la aplicación simplifica la lógica del programa. Todas las operaciones (alta, edición, eliminación y filtrado) actúan directamente sobre este array, y posteriormente la interfaz se actualiza en función de su contenido. De este modo, el array no sólo almacena información, sino que se convierte en la base sobre la que se construye la representación visual.

Trabajar con una matriz de objetos también facilita la escalabilidad del sistema. En caso de necesitar nuevas propiedades (por ejemplo, categoría o proveedor), bastaría con añadirlas al objeto sin modificar la estructura global del programa.

3. Uso de métodos avanzados de matrices

Una parte fundamental del desarrollo ha sido la utilización de métodos avanzados propios de las matrices en JavaScript. En lugar de emplear bucles tradicionales como *for*, se han utilizado métodos declarativos como *map*, *filter* y *reduce*, que permiten trabajar directamente sobre los datos del array de forma más expresiva y estructurada.

3.1. Método *.map()*

El método *.map()* se utiliza para transformar cada elemento del array *productos* en una representación HTML que posteriormente se inserta en la tabla del documento.

Fragmento representativo:

```
function renderizarTabla(lista) {
  cuerpoTabla.innerHTML = lista
  .map((p, indice) => {
    const total = (p.precio * p.cantidad).toFixed(2);
    return `
      <tr>
        <td>${p.nombre}</td>
        <td>${p.precio.toFixed(2)}</td>
        <td>${p.cantidad}</td>
        <td>${total}</td>
        <td>
          <button data-editar="${indice}" type="button">Editar</button>
          <button data-eliminar="${indice}" type="button">Eliminar</button>
        </td>
      </tr>
    `;
  })
  .join("");
}
```

Este método recorre la matriz y genera una nueva colección de cadenas HTML. Posteriormente, `.join("")` une todos los elementos en una única cadena que se inserta en el DOM. De esta manera, la tabla se reconstruye completamente cada vez que cambia el contenido del array, manteniendo la interfaz sincronizada con los datos.

3.2 Método *filter()*

El método *filter()* se utiliza dentro de la función *filtrarProductos* para generar una vista filtrada del inventario según el texto introducido por el usuario.

```
function filtrarProductos(consulta) {  
  const q = consulta.trim().toLowerCase();  
  if (!q) return productos;  
  return productos.filter(p => p.nombre.toLowerCase().includes(q));  
}
```

En primer lugar, la cadena introducida se normaliza eliminando espacios innecesarios y convirtiéndola a minúsculas. Si la búsqueda está vacía, se devuelve directamente el array completo.

En caso contrario, *filter()* recorre la matriz y devuelve un nuevo array con los productos cuyo nombre contiene la cadena buscada. De este modo, no se modifica la matriz original, sino que se genera una colección temporal que posteriormente se renderiza en la tabla.

3.3 Método *reduce()*

El método *reduce()* se utiliza para calcular el valor total del inventario a partir de los datos almacenados en la matriz.

Implementación en la aplicación:

```
function calcularGranTotal(lista) {  
  return lista.reduce((suma, p) => suma + p.precio * p.cantidad, 0);  
}
```

En este caso, *reduce()* recorre todos los elementos del array acumulando en la variable *suma* el resultado de multiplicar el precio por la cantidad de cada producto. El valor inicial del acumulador es 0.

Además, se incluye una prueba unitaria para verificar su funcionamiento:

```
function test_calcularGranTotal() {  
  const lista = [{ nombre: "A", precio: 10, cantidad: 2 }, { nombre: "B", precio: 20,  
cantidad: 1 }];  
  const total = lista.reduce((suma, p) => suma + p.precio * p.cantidad, 0);  
  console.assert(total === 40, "Fallo: reduce (gran total)");  
}
```

Esta prueba comprueba que el cálculo del total es correcto para un conjunto de datos controlado. Si el resultado no coincide con el valor esperado, la consola del navegador mostrará un error.

3.4 Relación entre los métodos utilizados

Aunque *map()*, *filter()* y *reduce()* cumplen funciones distintas, todos operan sobre la misma estructura de datos y permiten trabajar con la matriz de forma coherente y organizada.

map() transforma cada elemento del array en una nueva representación, en este caso filas HTML.

filter() selecciona únicamente aquellos elementos que cumplen una condición determinada, generando una vista parcial del inventario sin modificar los datos originales.

reduce() sintetiza la información del conjunto completo en un único valor acumulado.

En conjunto, estos métodos permiten aplicar transformaciones, selecciones y cálculos sobre los datos sin necesidad de utilizar bucles tradicionales. Esto facilita la lectura del código y refuerza la idea de que la matriz actúa como núcleo del funcionamiento de la aplicación.

4. Gestión del estado: modo edición y modo alta

Además de las operaciones básicas sobre la matriz, la aplicación incorpora un mecanismo de control para diferenciar entre el modo de alta y el modo de modificación de productos. Este comportamiento se gestiona mediante la variable *indiceEnEdicion*.

```
let indiceEnEdicion = null;
```

Cuando su valor es *null*, el formulario funciona en modo alta, es decir, al enviarlo se añade un nuevo producto al array. En cambio, cuando contiene un índice numérico, indica que el usuario está editando un producto existente.

Al pulsar el botón “Editar” de una fila, se ejecuta la carga de los datos del producto en el formulario y se actualiza el valor de *indiceEnEdicion*:

```
if (btnEditar) {  
  const indice = Number(btnEditar.dataset.editar);  
  const producto = productos[indice];  
  
  inputNombre.value = producto.nombre;  
  inputPrecio.value = producto.precio;  
  inputCantidad.value = producto.cantidad;  
  
  indiceEnEdicion = indice;  
  formularioProducto.querySelector('button[type="submit"]').textContent = "Actualizar";  
}
```

Posteriormente, al enviar el formulario, el sistema comprueba si *indiceEnEdicion* tiene un valor distinto de *null*. En ese caso, en lugar de añadir un nuevo elemento, se actualiza el producto correspondiente dentro del array.

Este mecanismo evita la creación de duplicados y permite reutilizar el mismo formulario para dos funciones distintas. De este modo, la variable *indiceEnEdicion* actúa como un indicador de estado que determina el comportamiento del sistema en cada momento.

Gestor de Productos

Nombre del producto

Precio

Cantidad

Añadir

Buscar productos

Escribe para filtrar por nombre...

Buscar

Consejo: usa el cuadro de búsqueda para filtrar productos por nombre.

Inventario

Nombre	Precio	Cantidad	Total	Acciones	
Teclado	39.99	10	399.90	Editar	Eliminar
Ratón	19.99	25	499.75	Editar	Eliminar
Monitor	149.99	6	899.94	Editar	Eliminar

Valor total del inventario: 1799.59

Figura 1. Formulario en modo alta.

Gestor de Productos

Nombre del producto

Precio

Cantidad

Actualizar

Buscar productos

Escribe para filtrar por nombre...

Buscar

Consejo: usa el cuadro de búsqueda para filtrar productos por nombre.

Inventario

Nombre	Precio	Cantidad	Total	Acciones	
Teclado	39.99	10	399.90	Editar	Eliminar
Ratón	19.99	25	499.75	Editar	Eliminar
Monitor	149.99	6	899.94	Editar	Eliminar

Valor total del inventario: 1799.59

Figura 2. Activación del modo edición tras seleccionar un producto.

Gestor de Productos

Nombre del producto: e.j. Teclado Precio: e.j. 39.99 Cantidad: e.j. 10 **Añadir**

Buscar productos

Escribe para filtrar por nombre... **Buscar**

Consejo: usa el cuadro de búsqueda para filtrar productos por nombre.

Nombre	Precio	Cantidad	Total	Acciones
Teclado	39.99	3	119.97	Editar Eliminar
Ratón	19.99	25	499.75	Editar Eliminar
Monitor	149.99	6	899.94	Editar Eliminar

Valor total del inventario: 1519.66

Figura 3. Actualización del producto y retorno al modo alta.

Las capturas anteriores muestran el ciclo completo del modo edición: activación del estado mediante la selección de un producto, modificación de los datos y posterior actualización del array. Tras realizar la actualización, la variable *indiceEnEdicion* vuelve a su valor inicial (*null*), lo que permite que el formulario recupere su comportamiento de alta. Este flujo garantiza que no se generen duplicados y que la interfaz permanezca sincronizada con la matriz de datos en todo momento.

5. Delegación de eventos

La aplicación utiliza delegación de eventos para gestionar las acciones de los botones “Editar” y “Eliminar”. En lugar de asignar un manejador de eventos individual a cada botón generado dinámicamente, se añade un único *addEventListener* al elemento *tbody* de la tabla.

Fragmento de código:

```
const btnEliminar = e.target.closest("button[data-eliminar]");
const btnEditar = e.target.closest("button[data-editar]");

if (btnEliminar) {
  const indice = Number(btnEliminar.dataset.eliminar);
  eliminarProducto(indice);
  refrescar();
  return;
}

if (btnEditar) {
```

```

const indice = Number(btnEditar.dataset.editar);
const producto = productos[indice];

inputNombre.value = producto.nombre;
inputPrecio.value = producto.precio;
inputCantidad.value = producto.cantidad;

indiceEnEdicion = indice;
formularioProducto.querySelector('button[type="submit"]').textContent = "Actualizar";
}
});

```

Este sistema permite capturar los clics sobre botones que no existían inicialmente en el DOM, ya que las filas de la tabla se generan dinámicamente mediante *map()*. Al delegar el evento en un elemento contenedor estable, se simplifica la gestión de eventos y se evita tener que reasignarlos cada vez que se renderiza la tabla.

6. Pruebas unitarias

Con el objetivo de verificar el correcto funcionamiento de las operaciones principales, se han incluido pruebas unitarias básicas utilizando *console.assert()*.

Estas pruebas permiten validar el comportamiento de la lógica del programa mediante conjuntos de datos controlados, comprobando que los resultados obtenidos coinciden con los valores esperados.

Bloque completo de pruebas implementadas:

```

/**
 * =====
 * PRUEBAS UNITARIAS
 * =====
 */

// Verifica que se puedan agregar elementos correctamente a una lista.
function test_agregarProducto() {
  let lista = [{ nombre: "A", precio: 1, cantidad: 1 }];
  lista = [...lista, { nombre: "B", precio: 2, cantidad: 2 }];
  console.assert(lista.length === 2, "Fallo: agregar (longitud)");
  console.assert(lista[1].nombre === "B", "Fallo: agregar (contenido)");
}

// Verifica que se puedan eliminar elementos correctamente de una lista.
function test_eliminarProducto() {
  let lista = [{ nombre: "A", precio: 1, cantidad: 1 }, { nombre: "B", precio: 2, cantidad: 2 }];
  lista = lista.filter((_, i) => i !== 0);
  console.assert(lista.length === 1, "Fallo: eliminar (longitud)");
  console.assert(lista[0].nombre === "B", "Fallo: eliminar (contenido)");
}

// Verifica que el filtrado por nombre funcione correctamente.

```

```
function test_filtrarProductos() {
  let lista = [{ nombre: "Teclado", precio: 1, cantidad: 1 }, { nombre: "Ratón", precio: 1,
cantidad: 1 }];
  const res = lista.filter(p => p.nombre.toLowerCase().includes("tec"));
  console.assert(res.length === 1, "Fallo: filtrar (longitud)");
  console.assert(res[0].nombre === "Teclado", "Fallo: filtrar (contenido)");
}

// Verifica que el cálculo del total general funcione correctamente.
function test_calcularGranTotal() {
  const lista = [{ nombre: "A", precio: 10, cantidad: 2 }, { nombre: "B", precio: 20,
cantidad: 1 }];
  const total = lista.reduce((suma, p) => suma + p.precio * p.cantidad, 0);
  console.assert(total === 40, "Fallo: reduce (gran total)");
}

test_agregarProducto();
test_eliminarProducto();
test_filtrarProductos();
test_calcularGranTotal();

console.log("Pruebas unitarias completadas correctamente.");
```

Estas pruebas se ejecutan en la consola del navegador. Si alguna condición no se cumple, se muestra un mensaje de error indicando el fallo correspondiente. En caso contrario, se confirma la correcta ejecución mediante el mensaje final en consola.



Figura 4. Ejecución de la aplicación junto con la consola mostrando la superación de las pruebas unitarias.

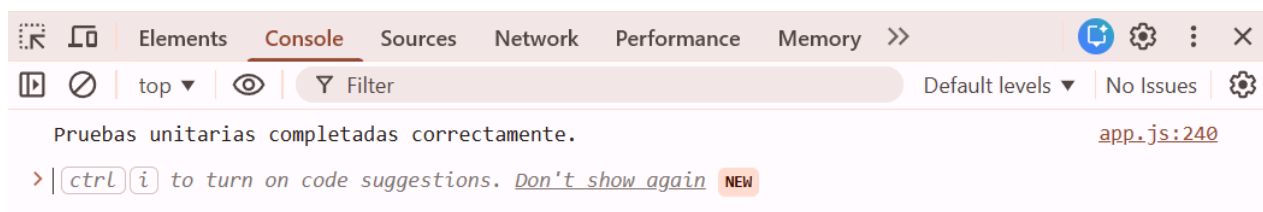


Figura 5. Resultado de las pruebas unitarias en la consola del navegador.

7. Conclusiones

El desarrollo de este caso práctico ha permitido aplicar de forma práctica las matrices en JavaScript como estructura central para la gestión de datos en el entorno cliente. A través de la utilización de métodos como *map*, *filter* y *reduce*, se han implementado transformaciones, búsquedas y cálculos agregados sobre una misma estructura de datos.

La incorporación de un mecanismo de control para el modo edición, junto con la delegación de eventos y la manipulación dinámica del DOM, ha permitido construir una aplicación interactiva y coherente sin recargar la página.

Finalmente, la inclusión de pruebas unitarias básicas refuerza la fiabilidad de la lógica implementada y facilita la verificación del comportamiento esperado durante el desarrollo.

8. Referencias

- **W3Schools**. “JavaScript Tutorial.” Disponible en: <https://www.w3schools.com/js/>
Consultado para revisar el uso de métodos de arrays como *map*, *filter* y *reduce*, así como ejemplos de manipulación del DOM y gestión de eventos en el entorno cliente.
- **Mimo**. **Plataforma de aprendizaje interactivo de programación.**
Utilizada como apoyo para reforzar la práctica con estructuras de datos en JavaScript, especialmente arrays y funciones, aplicadas posteriormente en el desarrollo del sistema de inventario.